

WDW.GC - User Manual

H.J. Brumatto, G.A. Monerat, F.B.S. Oliveira, E.V. Corrêa Silva,
A.L.B. Ribeiro, V.J. Monteiro, A.O. Castro Junior

October 4, 2026

Contents

I	User Guide	2
1	Downloading and running the program	3
1.1	System Architecture and Hardware Requirements	4
1.2	Building and Executing	4
1.2.1	Quick Start (Windows 1-Click)	4
1.2.2	Manual Build (Linux / macOS / HPC)	4
2	Normal use	6
2.1	The problem and Physical Motivation	6
2.2	Defining the parameters	7
2.3	Running the program	7
2.4	The Recovery Points	9
2.5	Extracting and Visualizing Results	10
2.6	Restrictions	11
II	Developer Guide and Physics Customization	13
3	Advanced Use	14
3.1	Changing the Effective Potential	14
3.2	Changing INI Parameters	14
3.3	Changing Initial Conditions (Ψ_0)	15
A	BLAS Library	17
A.1	Macros	17
A.2	Class: <code>sm_matrix<T></code>	17
A.2.1	Description	17
A.2.2	Public Methods	17
A.3	Code Organization	19

Abstract

This manual describes the use of the `wdw.gc` package, developed to solve the Wheeler–DeWitt equation using a Crank–Nicolson discretization combined with the Conjugate Gradient (CG) method. The package accompanies the article entitled

“WDW.GC: An Algorithm Based on the Crank–Nicolson (2+1) Method and the Conjugate Gradient Method for Solving the Wheeler–DeWitt Equation of the Early Universe”

by the same authors.

The purpose of this guide is to explain how to compile, configure, execute, and analyze simulations performed with the package.

Part I

User Guide

1. Downloading and running the program

This manual refers to the program `wdw.gc`, which implements a Crank–Nicolson discretization together with the Conjugate Gradient (CG) method to solve the Wheeler–DeWitt equation describing the quantum evolution of the early Universe during the Planck era.

The source code is available from the GitHub repository:

`https://github.com/brumatto/wdw.gc.git`

or can be cloned directly using

```
gh repo clone brumatto/wdw.gc
```

The files are:

SMmatrix.hpp Sparse matrix library providing the linear algebra operations required by the Conjugate Gradient solver.

GCsolve.hpp, GCSolve.cpp Implementation of the Conjugate Gradient (CG) algorithm for solving sparse linear systems.

wdw.gc.aux.cpp Auxiliary routines responsible for reading the configuration file and initializing the simulation parameters.

wdw.gc.hpp, wdw.gc.cpp Main program responsible for reading the INI configuration file, building the numerical problem, and solving the Wheeler–DeWitt equation.

wdw.ini Example configuration file containing a complete simulation setup.

fast.ini Lightweight configuration intended for quick validation and low-memory testing.

CMakeLists.txt CMake configuration used to build the project on Windows, Linux, and macOS.

run.bat Windows script that automatically compiles and executes the program.

wdw.gc.manual.pdf This manual

1.1 System Architecture and Hardware Requirements

To help visualize how the program processes data, the basic workflow of `wdw.gc` follows this pipeline:

INI file $\longrightarrow$ Crank–Nicolson discretization
 $\longrightarrow$ Sparse matrices (SMmatrix)
 $\longrightarrow$ CG solver
 $\longrightarrow$ DAT files

Since the solver stores several large sparse matrices in memory, the amount of available RAM is the primary hardware limitation. Table 1.1 provides recommended memory requirements for typical grid sizes.

Mesh Size ($N_a \times N_\varphi$)	RAM Required	Recommended Use
100×150	~ 16 MB	Quick testing and debugging (<code>fast.ini</code>)
1000×1500	~ 1.5 GB	Typical workstation or high-end laptop
6000×9000	~ 16 GB	HPC / Production runs (<code>wdw.ini</code>)

Table 1.1: Hardware requirements based on grid size.

1.2 Building and Executing

1.2.1 Quick Start (Windows 1-Click)

For Windows users, the repository includes an automation script that compiles and executes the program with a single click. No manual CMake configuration is required.

1. Edit your physical parameters in `examples/fast.ini` or `examples/wdw.ini`.
2. Double-click the `run.bat` file located in the root directory.

The script will automatically set up the build environment, compile the code in the background, and start the simulation.

1.2.2 Manual Build (Linux / macOS / HPC)

If you are on Linux, macOS, or prefer full control, build the project manually using CMake:

```
1 mkdir build && cd build
2 cmake ..
3 cmake --build . --config Release -j
```

It is recommended to first test the configuration using a small grid (for example, `fast.ini`) before running large production simulations.

```

C:\WINDOWS\system32\cmd. x + v
[*] Starting Simulation...

=====
Wheeler-DeWitt Solver (Crank-Nicolson + CG)
=====

[1/6] Reading configuration parameters...
-> Using 16 OpenMP threads.
[2/6] Allocating memory and calculating Effective Potential...
-> Grid: 1800 x 1500 points.
[3/6] Building sparse matrices H and B...
-> (Warning: This step may take a few minutes due to the grid size)
[4/6] Calculating auxiliary matrices...
-> Transposed matrix calculated.
-> H matrix (M*M) calculated.
[5/6] Initializing Wave Function (Psi)...
-> Initial Psi generated and saved in: dados_teste/teste_00.000.dat

[6/6] Starting Time Evolution...
t      Psi2      Psi2_a      Psi2_b      Psi2_in      amed      amed_a      amed_b      amed_in      phimed      phimed_a      phimed_b      phimed_in      phi2m      phi2m_a      phi2m_b      phi2m_in
0.001000  1.000000  0.000000  0.000000  1.000000  0.912152  0.170141  1.004196  0.1701410.178534  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.031250
0.031250  0.031250  0.031250  0.031250  1.000000  0.912152  0.170141  1.004196  0.1701410.178534  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.031250
Iniciando itera[*]aes...
norma = 0.0202017; err = 0.0201987; Psi_m 3.51188
norma = 0.1819e-06; err = 0.1819e-06; Psi_m 3.51188
norma = 1.6777e-06; err = 1.6775e-06; Psi_m 3.51188
norma = 4.23916e-08; err = 3.93921e-08; Psi_m 3.51188
norma = 5.19980e-09; err = 4.08657e-09; Psi_m 3.51188
-> Step 0 completed: Error = 5.19980e-09 (Solved in 5 CG iterations)
0.002000  1.000000  0.000000  0.000000  1.000000  0.912134  0.170146  1.004197  0.1701460.178539  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.031253
0.031250  0.031250  0.031250  0.031250  1.000000  0.912134  0.170146  1.004197  0.1701460.178539  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.031253
Iniciando itera[*]aes...
norma = 0.0201957; err = 0.0201866; Psi_m 3.51113
norma = 0.17554e-06; err = 0.17554e-06; Psi_m 3.51114
norma = 1.67651e-06; err = 1.67612e-06; Psi_m 3.51114
norma = 4.25233e-08; err = 3.94070e-08; Psi_m 3.51114
norma = 5.24635e-09; err = 4.87812e-09; Psi_m 3.51114
-> Step 1 completed: Error = 5.24635e-09 (Solved in 5 CG iterations)

[*] Checkpoint saved: dados_teste/teste_00.002.dat
0.003000  1.000000  0.000000  0.000000  1.000000  0.912082  0.170159  1.004199  0.1701590.178557  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.031281
0.031281  0.031281  0.031281  0.031281  1.000000  0.912082  0.170159  1.004199  0.1701590.178557  0.000000  0.000000  0.000000  0.000000  0.000000  0.000000  0.031281
Iniciando itera[*]aes...
norma = 0.0201775; err = 0.0201668; Psi_m 3.50989
norma = 0.15016e-06; err = 0.15016e-06; Psi_m 3.50999
norma = 1.67462e-06; err = 1.67344e-06; Psi_m 3.50999
norma = 4.26969e-08; err = 3.93451e-08; Psi_m 3.50999
norma = 5.28154e-09; err = 5.0134e-09; Psi_m 3.50999

```

Figure 1.1: Example of automated execution using the run.bat script on Windows.

2. Normal use

2.1 The problem and Physical Motivation

The primary utility of the `wdw.gc` program is to simulate the quantum origins of the early Universe during the Planck Era. By solving the Wheeler DeWitt equation, the software outputs the complex wave function $\Psi(a, \varphi, \tau)$.

Physically, the squared magnitude $|\Psi|^2$ generated in the output `.dat` files represents the probability density of the Universe existing with a specific scale factor a and scalar field φ at a given dynamical time τ . This allows researchers to study quantum tunneling effects and the emergence of classical spacetime.

The numerical model solves the following Wheeler–DeWitt equation:

$$\left[-\frac{1}{12} \frac{\partial^2}{\partial a^2} + \frac{1}{2} \frac{\partial^2}{\partial \varphi^2} + V_{eff}(a, \varphi) \right] \Psi(a, \varphi, \tau) = i \frac{\partial}{\partial \tau} \Psi(a, \varphi, \tau)$$

where a is the scale factor of the universe, φ is the massive scalar field conformally coupled to gravity, τ is the dynamical variable associated with the radiation-based perfect fluid. V_{eff} is the effective potential that provides a barrier to the wave function Ψ of the universe. The effective potential used is:

$$V_{eff}(a, \varphi) = 3a^2 - \Lambda a^4 - \frac{1}{2} \varphi^2 - \frac{1}{2} m^2 a^2 \varphi^2$$

where Λ is Einstein’s cosmological constant representing the vacuum energy of the inflation field, and m is a mass term associated with the effective potential.

The following homogeneous Dirichlet boundary conditions are imposed:

$$\begin{cases} \Psi(a = 0, \varphi, \tau) = 0, \\ \Psi(a = \infty, \varphi, \tau) = 0, \\ \Psi(a, \varphi = -\infty, \tau) = 0, \\ \Psi(a, \varphi = \infty, \tau) = 0 \end{cases}$$

The initial wave packet is defined as for $\Psi(a, \varphi, 0)$ is:

$$\Psi(a, \varphi) = \frac{2^{15/4}}{\sqrt{\pi}} E_a^{3/4} E_\varphi^{1/4} a \exp(-4E_a a^2 - 2E_\varphi \varphi^2)$$

where E_a and E_φ are energies associated with the scale factor and scalar field respectively.

2.2 Defining the parameters

To provide a quick reference, Table 2.1 summarizes the most important parameters found in the `.ini` file:

Table 2.1: Quick Reference for main INI parameters.

Parameter	Example	Description
<code>Na</code>	6000	Number of mesh points for the scale factor a .
<code>Nphi</code>	9000	Number of mesh points for the scalar field φ .
<code>Lambda</code>	1.5	Cosmological constant Λ .
<code>DeltaT</code>	0.00001	Time step $\Delta\tau$ for temporal evolution.
<code>Print</code>	2500	Number of calculation steps between saving <code>.dat</code> snapshots.
<code>BaseDir</code>	<code>data/run1</code>	Output directory for generated <code>.dat</code> files.

The physical evolution of the wave function is determined by four parameters: Λ (`Lambda`), m (`m`), E_a (`Ea`) and E_φ (`Ephi`). These parameters can be set to a specific value for each test case in an INI file.

Unless another file is explicitly specified, the program searches for `wdw.ini` in the current working directory. Alternatively, a configuration file may be provided from the command line:

```
./wdw.gc ../examples/wdw.ini
```

In addition to the physics parameters, the INI file allows the user to configure numerical grid, execution, and output settings. Table 2.2 provides a comprehensive description of all available parameters.

Table 2.3 shows an example of an INI file. Any line starting with `#` or any blank line is ignored. The program expects to read lines containing a pair: `parameter = value`. The parameters are case-sensitive, and in this example, we can see a complete initialization setup.

2.3 Running the program

Executing a simulation follows a straightforward workflow. Once your hardware is provisioned, follow these steps:

1. **Configure the INI file:** Copy an existing template (e.g., `wdw.ini`) and set your physical parameters, grid size, and desired `BaseDir` for the output.
2. **Create the output directory:** Ensure the folder specified in `BaseDir` exists in your file system before running the program, otherwise, the data dumping will fail.
3. **Execute:** Run the program passing the INI file as an argument:

```
./wdw.gc path/to/your/config.ini
```

Table 2.2: Description of some of the INI configuration parameters.

INI Key	Symbol	Description
a	a_∞	Numerical infinity for the scale factor range.
phi	$\pm\varphi_\infty$	Numerical infinity for the scalar field range.
Na	N_a	Mesh points for the scale factor ($\Delta a = a_\infty/N_a$).
Nphi	N_φ	Mesh points for the scalar field ($\Delta\varphi = 2\varphi_\infty/N_\varphi$).
m	m	Mass of the scalar field.
Lambda	Λ	Cosmological constant Parameter.
Ea	E_a	Energy associated with Scale Factor.
Ephi	E_φ	Energy associated with Scalar Field.
DeltaT	$\Delta\tau$	Step size for each iteration in the temporal evolution.
Tlast	–	Final value of τ to mark the end of the evolution.
Print	–	Number of time steps between successive output snapshots.
rest	–	Continuation time associated with a specific recovery point.
BaseDir	–	Directory path to save the recovery point files.
file	–	Prefix for the recovery point files (saved in the format <code>prefix.00.000</code>).
Resume	–	Enables restarting from a previously saved checkpoint.
aVmax	$a_{V_{\max}}$	Value of a where the effective potential has its maximum peak (used for tunneling and contour surface).
Imax	$i_{\max}$	Maximum number of iterations for the Conjugate Gradient algorithm before aborting.
Emax	$e_{\max}$	Maximum error norm for $\Psi^n - \Psi^{n-1}$ accepted as convergence in the CG algorithm.
NProc	–	Number of OpenMP threads. If omitted, all available hardware threads are used.

Understanding the Console Output:

As the program runs, it provides real-time feedback:

- **Standard Output (stdout):** Displays the time step τ followed by average quantum expectations: Ψ^2 , $\langle a \rangle$, $\langle \varphi \rangle$, and $\langle \varphi^2 \rangle$. These are integrated over four distinct intervals (Full, after CTs, before CTs, and within V_{ef}).
- **Error Output (stderr):** Displays initialization diagnostics (threads, matrices) and real-time Conjugate Gradient convergence metrics per iteration (norm error and max $|\Psi^2(t_i) - \Psi^2(t_{i+1})|$).

2.4 The Recovery Points

During execution, the program periodically saves the state of the wave function Ψ as a recovery point (snapshot). These files serve a dual purpose: they provide data for later cosmological analysis and act as checkpoints to resume interrupted executions.

Saving Mechanism and File Naming

Snapshots are saved in the directory specified by the `BaseDir` parameter. The frequency of these saves is determined by the `DeltaT` and `Print` parameters: `DeltaT` sets the step size for the temporal evolution of τ , and a snapshot is written every `Print` steps.

The generated files follow a strict naming convention:

`<file>_<τ>.dat`

Where `<file>` is the prefix defined in the INI file, and `<τ>` is the time position formatted as `##.###`. For example, using the parameters from Table 2.3, the first four recovery points would be:

- `a5phi4_00.000.dat` ($\tau = 0.0$)
- `a5phi4_00.025.dat` ($\tau = 0.025$)
- `a5phi4_00.050.dat` ($\tau = 0.050$)
- `a5phi4_00.075.dat` ($\tau = 0.075$)

Binary File Structure and Grid Mapping

To optimize storage, and specially to avoid introducing rounding errors into the processing continuity, and I/O performance, each file is written in a raw binary format containing $(N_a \times N_\varphi) + 1$ pairs of double-precision floating-point numbers. Each pair corresponds to the real and imaginary parts of Ψ at a specific grid point.

The spatial grid is defined by the resolution parameters (`Na`, `Nphi`) and the numerical infinity boundaries (a_∞ , φ_∞). The step sizes between grid points are calculated as $\Delta a = a_\infty / N_a$ and $\Delta \varphi = 2\varphi_\infty / N_\varphi$.

The coordinates (a_i, φ_j) for any discrete wave function state $\Psi_{i,j} = \Psi(a_i, \varphi_j)$ are given by:

$$\begin{aligned} a_i &= a_0 + i\Delta a \\ \varphi_j &= \varphi_0 + j\Delta\varphi \end{aligned}$$

where the grid boundaries are initialized as:

$$a_0 = 0, \quad a_{N_a} = a_\infty, \quad \varphi_0 = -\varphi_\infty, \quad \varphi_{N_\varphi} = \varphi_\infty$$

When written to the file, the 2D grid is flattened into a 1D sequence in row-major order:

$$\Psi_{0,0}, \Psi_{1,0}, \Psi_{2,0}, \dots, \Psi_{N_a,0}, \Psi_{0,1}, \Psi_{1,1}, \dots, \Psi_{N_a,N_\varphi}$$

Resuming an Execution

To continue a simulation from a specific snapshot rather than starting from the initial condition $\Psi(\tau = 0)$, the following parameters must be configured in the INI file:

- **Resume:** Set to `true`.
- **rest:** Specify the exact τ value to resume from (e.g., 0.075).
- **resfile:** Provide the exact filename of the target recovery point located in `BaseDir`.

2.5 Extracting and Visualizing Results

The ultimate goal of the simulation is to analyze the evolution of the wave packet. Since `wdw.gc` outputs raw binary data arrays to maximize HPC performance and save disk space, you must post-process the `.dat` files to extract physical meaning.

Python Visualization Example:

The easiest way to extract and visualize the probability density surface is using Python (`numpy` and `matplotlib`). Ensure your grid dimensions in the script exactly match the `Na` and `Nphi` parameters from your INI file:

Listing 2.1: Python script to read and plot wave function data

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 1. Define mesh parameters (must match INI file)
5 Na, Nphi = 3000, 4500
6 filename = "lbd1.5/m0.1/a5phi4/a5phi4_00.000.dat"
7
8 # 2. Extract binary data (complex doubles)
9 # The array size is (Nphi + 1) rows by (Na + 1) columns
10 data = np.fromfile(filename, dtype=np.complex128)
11 psi = data.reshape((Nphi + 1, Na + 1))
12
13 # 3. Plot the absolute density |Psi|^2
```

```

14 plt.imshow(np.abs(psi)**2, origin='lower', aspect='auto')
15 plt.colorbar(label=r'$|\Psi|^2$ (Probability Density)')
16 plt.title("Wave Packet Density of the Universe")
17 plt.xlabel("Scale Factor (a)")
18 plt.ylabel(r"Scalar Field (\varphi)")
19 plt.show()

```

2.6 Restrictions

This program was created to make use of an HPC (High Performance Cluster), its speedup shows that the best performance occurs using $30 \sim 50$ threads. The main restriction on its use is the memory size.

When running the program, it creates approximately 25 “vectors” of size $N_a \times N_\varphi$ of complex values (two *doubles*). For $N_a \times N_\varphi \approx 10^{10}$ approximately 128GBytes of RAM are required. It is not convenient to use a grid with a dimension larger than this.

It is not possible to change the boundary conditions for the PDE. We choose Ψ for $a = 0$, a_∞ and $\pm\varphi_\infty$ is defined as $\Psi = 0$ and is intrinsically defined in the code. To use something like $\left. \frac{\partial \Psi}{\partial a} \right|_{a=0} = 0$ or in any other contour, it will be necessary to develop another code.

```

# =====
# Wheeler-DeWitt Solver (wdw.gc) - Configuration File
# =====
# GENERAL RULES:
# - Lines starting with '#' and blank lines are ignored.
# - Variables are CASE SENSITIVE.
# - Format required: <VariableName> <Value>
# - Unrecognized or unused variables are safely ignored.
# =====

# --- Hardware / Parallelism ---
# NProc          40          # Uncomment to manually set OpenMP threads
#(defaults to max available)

# --- Grid Discretization (Crank-Nicolson) ---
Na              3000          # Number of grid points for the scale factor 'a'
Nphi            4500          # Number of grid points for the scalar field 'phi'
a               20            # Maximum boundary value for the scale factor grid
phi             40            # Maximum boundary value for the scalar field grid

# --- Effective Potential (Vef) ---
Lambda          1.5           # Cosmological constant parameter
m               0.1           # Mass of the scalar field
aVmax           1.0           # Cutoff or maximum limit for the potential

# --- Initial Wave Function (Psi0) ---
Ea              5.498         # Initial energy/momentum parameter for 'a'
Ephi            4.0           # Initial energy/momentum parameter for 'phi'

# --- Conjugate Gradient (CG) Solver ---
Imax            500           # Maximum number of CG iterations per time step
Emax            1e-8          # Error tolerance for CG convergence

# --- Time Evolution ---
Tlast           1.0           # Final time for the evolution
T               1.0           # Grid definition for time evolution Nt = T/DeltaT
DeltaT          0.0001        # Time step size for the evolution
Print           2             # Print/Save output every 'Print' iterations

# --- Checkpoints & Output Data ---
Resume          false         # Set to 'true' to resume from a previous checkpoint
rest            0.755         # Time point of the restored state (if Resume is true)
resfile         a5phi4_00.755 # Name of the checkpoint file to load
file            a5phi4_       # Prefix for generated output files
BaseDir         lbd1.5/m0.1/a5phi4 # Base directory to save output data

```

Table 2.3: INI file sample

Part II

Developer Guide and Physics Customization

3. Advanced Use

3.1 Changing the Effective Potential

By default, the effective potential is defined as:

$$V_{\text{eff}}(a, \varphi) = 3a^2 - \Lambda a^4 - \frac{1}{2}\varphi^2 - \frac{1}{2}m^2 a^2 \varphi^2$$

The implementation is not restricted to this particular form. Any effective potential compatible with the Wheeler–DeWitt equation can be implemented by modifying the source code. For example, consider a modified potential incorporating a new term V_0 :

$$V_{\text{eff}}(a, \varphi) = 3a^2 - \Lambda a^4 + \frac{aV_0}{\cosh a} - \frac{1}{2}\varphi^2 - \frac{1}{2}m^2 a^2 \varphi^2$$

Although the program defaults to the first potential, implementing an alternative potential requires only minor modifications to the source code. In the file `wdw.gc.cpp`, the potential function is implemented in the function defined `potential(...)`, located in `wdw.gc.cpp`. The function is evaluated during the construction of the effective potential matrix.

```
25 double potential(double a, double phi, double lambda, double mass) {  
26     double a2 = a*a;  
27     double phi2 = phi*phi;  
28     return (3*a2 - lambda*a2*a2 - phi2/2 - mass*mass*a2*phi2/2);  
29 }  
30 // ...  
119 Vef[j*(Na+1)+i] = potential(ai, phij, lambda, mass);
```

The variables `ai` and `phij` represent a_i and φ_j correspond to the coordinates of the current grid point. To change the effective potential, simply update the mathematical expression inside the `potential` function and ensure its call matches any new parameters.

If your new potential requires additional parameters (such as V_0), the parameter must also be added to the configuration parser, as described in Section 3.2. Once the changes are made, recompile the code using CMake. **Note:** When using Makefiles or Ninja, the build can be parallelized by passing the appropriate parallel-build option (for example, `-j` with GNU Make).

3.2 Changing INI Parameters

If a customized effective potential requires a new parameter (like V_0), you must declare it in both the configuration INI file and the C++ source code.

First, define the parameter name in the INI file (e.g., `V0`) and assign it a value. For readability, it is recommended to place the parameter together with the other effective-potential parameters.

```
1 # Vef:
2 Lambda 1.5
3 m 0.1
4 aVmax 1.0
5 V0 1.0
```

Next, declare the corresponding C++ variable. In `wdw.gc.cpp`, you will find the declarations for `lambda` and `mass`. You can add your new variable here:

```
39 double lambda, mass, v0;
```

Finally, extend the INI parser (lines 58 to 84) to read the new parameter from the INI file:

```
71 // Vef
72 else if (variavel == "Lambda") iss >> lambda;
73 else if (variavel == "m") iss >> mass;
74 else if (variavel == "aVmax") iss >> avmax;
75 else if (variavel == "V0") iss >> v0;
```

After saving these changes, recompile the code using CMake, ensuring that the OpenMP flags are enabled during compilation and linking.

3.3 Changing Initial Conditions (Ψ_0)

The default initial condition for Ψ_0 is based on a wave packet of the form:

$$\Psi_0 = Ca \exp(-Aa^2 - B\varphi^2) \quad (3.1)$$

The constants A , B , and C are constrained by the following physical requirements (normalization and expected energies):

$$\begin{aligned} \int_{-\infty}^{\infty} \int_0^{\infty} \Psi^* \Psi da d\varphi &= 1 \\ \int_{-\infty}^{\infty} \int_0^{\infty} \Psi^* \left(\frac{p_a^2}{12} \right) \Psi da d\varphi &= E_a \\ \int_{-\infty}^{\infty} \int_0^{\infty} \Psi^* \left(\frac{p_\varphi^2}{2} \right) \Psi da d\varphi &= E_\varphi \end{aligned}$$

where p_a and p_φ are the canonical momenta associated with the scale factor and the scalar field, respectively.

If you wish to use a different initial wave packet, such as:

$$\Psi_0 = Ca^2 \exp(-Aa^2 - B\varphi^2) \quad (3.2)$$

the values of the constants A , B , and C will change accordingly. This modification is handled directly within the source code.

The file `wdw.gc.aux.cpp` contains the implementation of the `createPsi0` function. You must update the mathematical logic for the initial conditions inside this function. If your modifications require changing the function protocol (its parameters), you must also update its declaration in the `wdw.gc.hpp` header file and its function call in `wdw.gc.cpp`:

```
132 // Generating the initial wave function
133 psi = createPsi0(Na, Nphi, inftyA, inftyphi, Ea, Ephi);
```

A. BLAS Library

Included among the project files is the header `SMmatrix.hpp`. The file `SMmatrix.hpp` implements a lightweight sparse linear algebra library inspired by the BLAS programming model. It is implemented as a template class, containing all the necessary methods, functions, and macros to efficiently perform sparse linear algebra operations.

A.1 Macros

- `#define ZERO (1e-300)`: We define a value as zero if its absolute magnitude is less than or equal to 10^{-300} . This threshold is necessary because floating-point arithmetic is subject to rounding errors, making exact comparisons unreliable. This definition is also crucial when working with complex numbers.

A.2 Class: `sm_matrix<T>`

A.2.1 Description

```
template<class T>
class sm_matrix<T>
```

A cell of the matrix is represented by a `std::pair<int, T>`. Each cell carries its column index j as the first element of the pair, while its associated data value is the second element. To preserve the sparse nature of the structure, if a cell's value evaluates to 0 (absolute value strictly less than `ZERO`), it is removed from memory. A single row of the matrix is represented as a vector of these cells (`vector<pair<int,T>>`).

The `sm_matrix` structure is constructed as a vector of rows, making the entire matrix a `vector<vector<pair<int,T>>>`. A cell (i, j) has its row index i defined by its position in the outer vector, while its column index j is extracted from the first element of the cell pair itself.

A.2.2 Public Methods

Table A.1 outlines the public methods available in the `sm_matrix<T>` class.

Table A.1: Public methods of the `sm_matrix<T>` class.

Signature	Details
<code>sm_matrix(int n)</code>	Description: Constructor. Initializes a sparse square matrix of dimension n with empty row vectors. Parameters: n (the dimension of the sparse square matrix).
<code>int size()</code>	Description: Returns the size n of the sparse square matrix ($n \times n$). Returns: The matrix dimension.
<code>void setCell(int i, int j, T value)</code>	Description: Assigns the value <code>value</code> to cell (i, j) . Parameters: i (row index), j (column index), and <code>value</code> . If the absolute value is strictly less than <code>ZERO</code> , the cell is deleted from the structure.
<code>T getCell(int i, int j)</code>	Description: Retrieves the value stored in cell (i, j) . Parameters: i (row index) and j (column index). Returns: The value of the cell, or 0 if it is not allocated in the matrix.
<code>void swapLine(int i, int j)</code>	Description: Swaps rows i and j within the matrix. Parameters: i and j (the indices of the rows to be exchanged).
<code>vector<pair<int,T>> line(int i)</code>	Description: Retrieves all allocated elements of a specific row. Parameters: i (the index of the row). Returns: A vector containing the cells of row i .
<code>void setline(int i, vector<pair<int,T>> line)</code>	Description: Assigns a complete vector of cells to a specific row. Parameters: i (the row index) and <code>line</code> (the vector of cells to be initialized as the row).
<code>vector<pair<int,T>>::iterator begin(int i)</code>	Description: Provides the starting iterator for the cells in a specific row. Parameters: i (the row index). Returns: An iterator pointing to the beginning of the row vector.

A.3 Code Organization

The `wdw.gc` project is organized into a small number of source files, each responsible for a specific part of the numerical solver. Figure A.1 summarizes the relationship between these components.

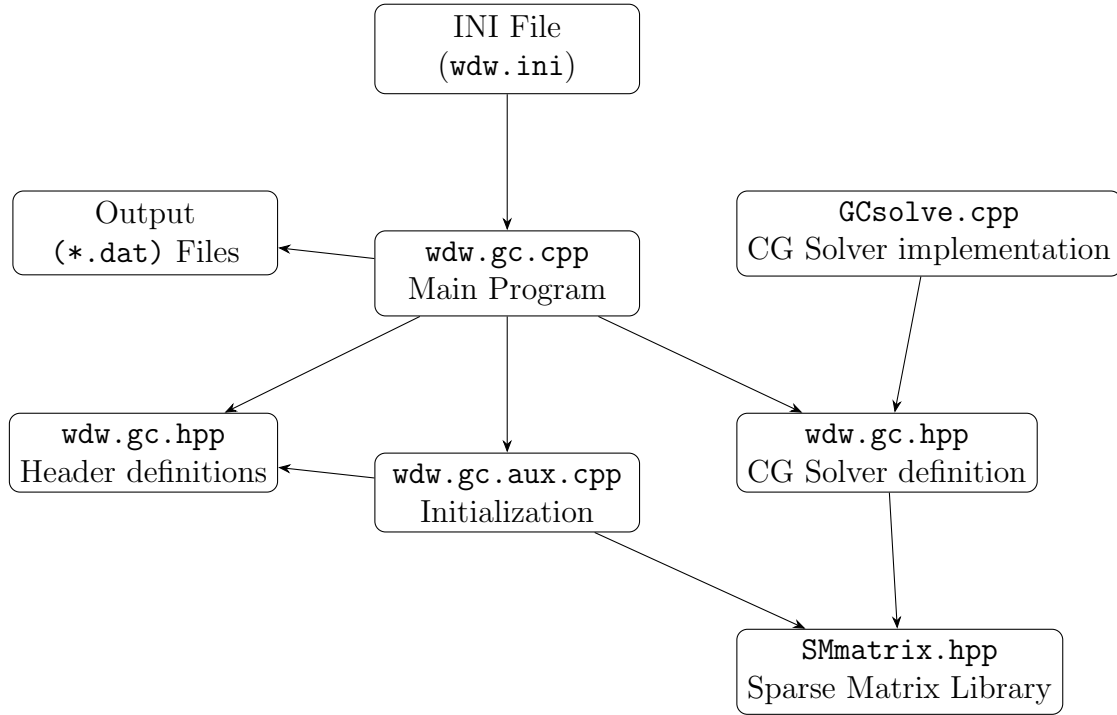


Figure A.1: Organization of the main source code modules.

The responsibilities of each file are summarized below:

- `wdw.gc.cpp`: main program. Reads the configuration “ini” file, initializes the simulation, assembles the numerical problem, and controls the time evolution. And outputs data: mean values and “.dat” checkpoint files.
- `wdw.gc.hpp`: declarations of shared functions, constants, and global variables.
- `wdw.gc.aux.cpp`: auxiliary routines, including initialization of the wave function and helper numerical functions.
- `GCsolve.hpp`: declarations of the Conjugate Gradient iterative solver functions.
- `GCsolve.cpp`: implementation of the Conjugate Gradient iterative solver functions.
- `SMmatrix.hpp`: sparse matrix data structure and associated linear algebra operations.